Persistencia de objetos con JSONB

1. Persistencia de objetos con JSONB

Persistencia de objetos con JSONB

Hasta ahora, cada atributo de tus entidades se almacenaba normalmente en una columna independiente. Este apartado introduce una posibilidad intermedia entre el modelo relacional clásico y las bases de datos orientadas a objetos: utilizar capacidades objeto-relacionales de PostgreSQL para almacenar datos estructurados dentro de una columna.

Con jsonb, PostgreSQL no guarda directamente un objeto Java ni sus métodos. Guarda su representación como un documento JSON, que Hibernate puede convertir al escribir y reconstruir al leer.

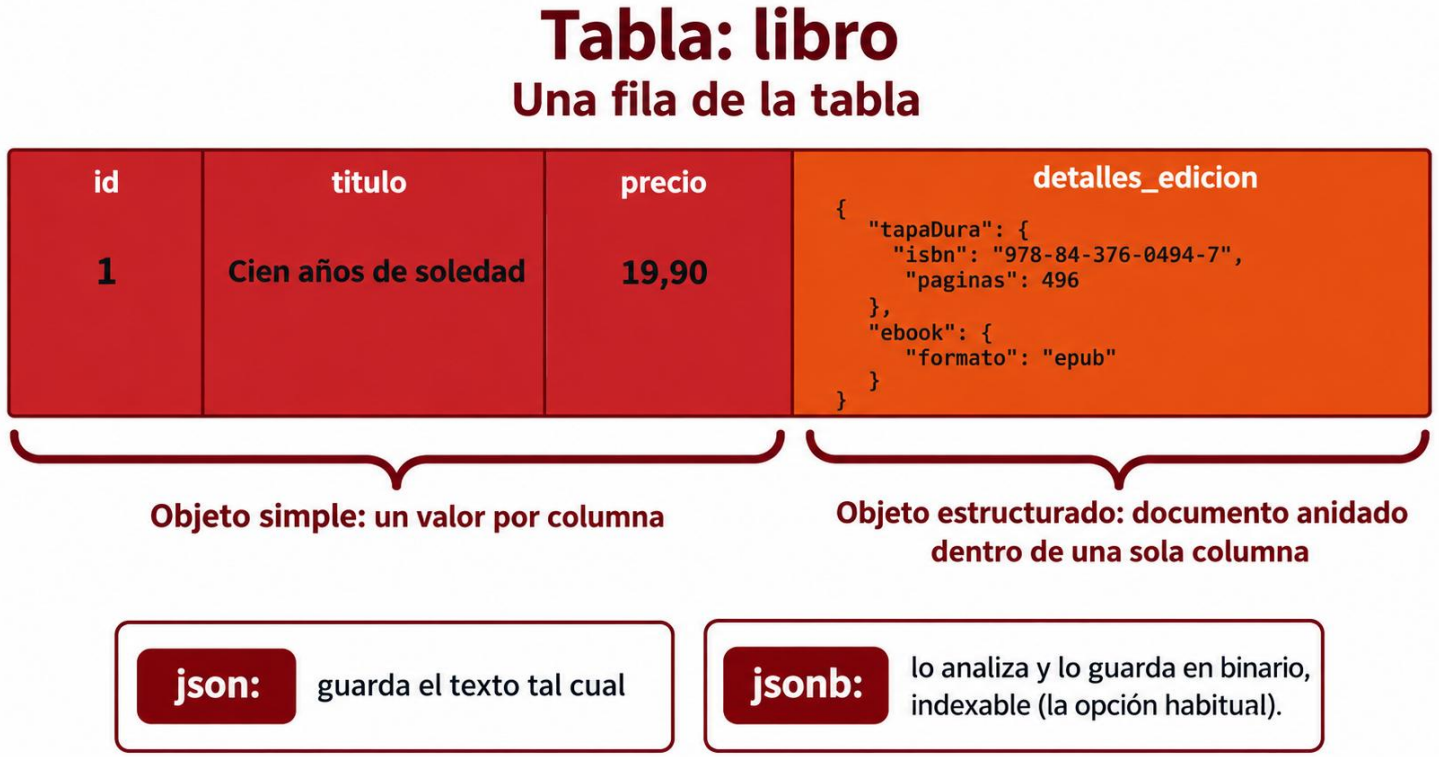
Repaso rápido: qué es JSON

JSON (JavaScript Object Notation) es un formato de texto para representar datos estructurados: objetos (pares clave-valor entre {}), arrays (listas entre []), y valores anidables entre ambos:

```
{
   "tapaDura":{
      "isbn":"978-84-376-0494-7",
      "paginas":496
   },
   "ebook":{
      "formato":"epub"
   }
}
```

Se ha convertido en el formato universal de intercambio de datos porque es legible por humanos, ligero, y prácticamente todos los lenguajes lo leen y escriben de forma nativa.

Objeto simple, objeto estructurado y jsonb



jsonb (JSON binario) permite que una fila normal, de una tabla normal, lleve dentro un objeto JSON completo —con estructura que puede variar de una fila a otra— conviviendo con las columnas y claves foráneas de siempre.

json o jsonb, y ninguna conexión nueva

json vs. jsonb

json guarda el texto tal cual, sin analizarlo. jsonb lo analiza y lo guarda en un formato binario, indexable y más rápido de consultar.

Por eso jsonb es casi siempre la opción preferida, salvo que necesites preservar el texto original exacto (espacios, orden de claves).

Sin base de datos aparte

No hace falta una base de datos documental (eso lo verás en el Tema 3) ni una conexión distinta.

Sigue siendo el mismo PostgreSQL y el mismo DataSource del Tema 1, gestionado exactamente igual: una columna jsonb no abre ni cierra nada por su cuenta.

Un ejemplo completo: detallesEdicion

Cada libro puede existir en varios formatos (tapa dura, bolsillo, ebook), cada uno con sus propios datos: un objeto estructurado dentro de la entidad Libro.

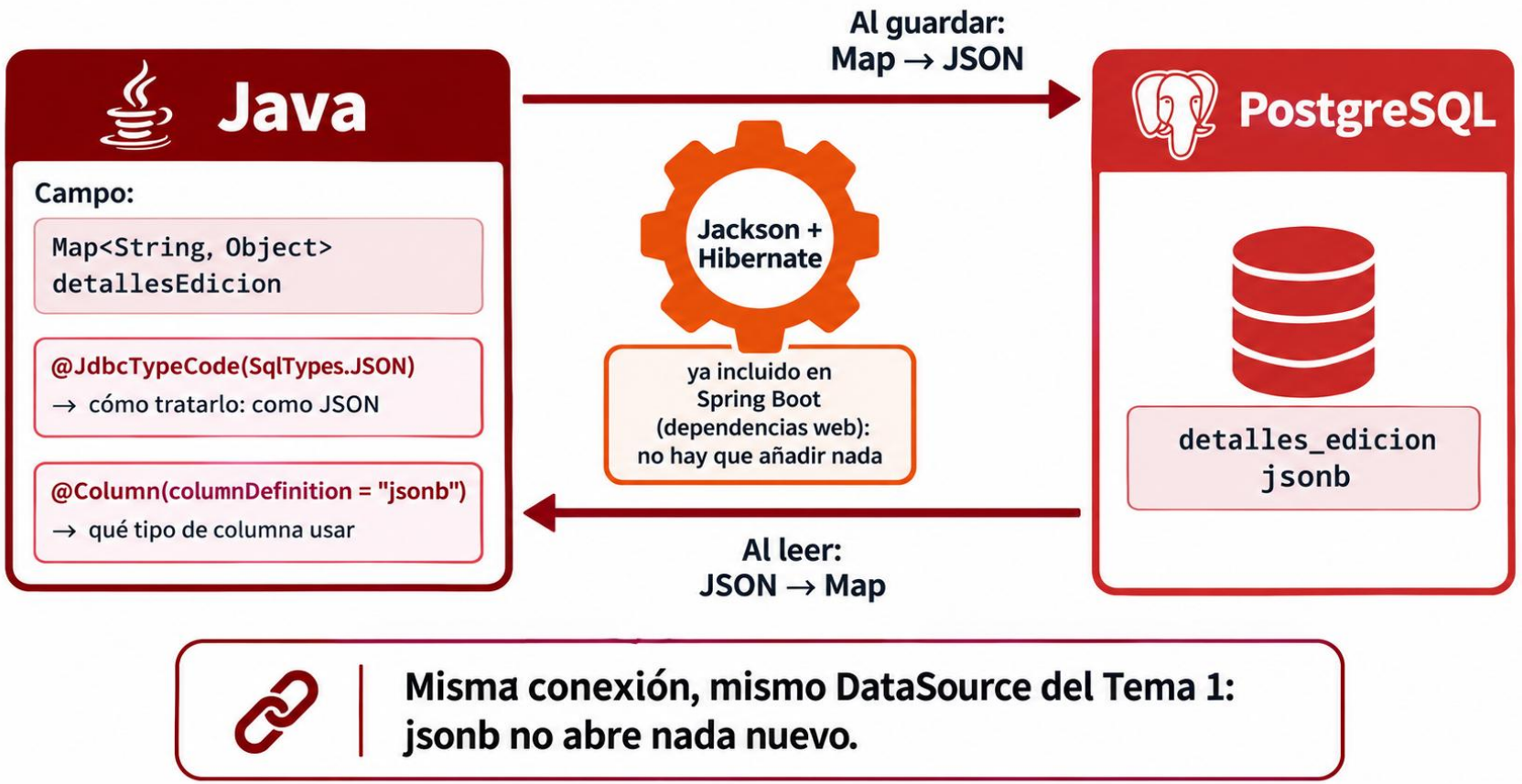
```
@Entity
@Table(name = "libro")
public class Libro {

    // ... id, titulo, precio, fechaPublicacion ...

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(columnDefinition = "jsonb")
    private Map<String, Object> detallesEdicion;
}
```

@Column(columnDefinition = "jsonb") le dice a Hibernate qué tipo de columna PostgreSQL usar; @JdbcTypeCode(SqlTypes.JSON) le dice cómo tratar ese campo: como JSON, no como un texto plano cualquiera.

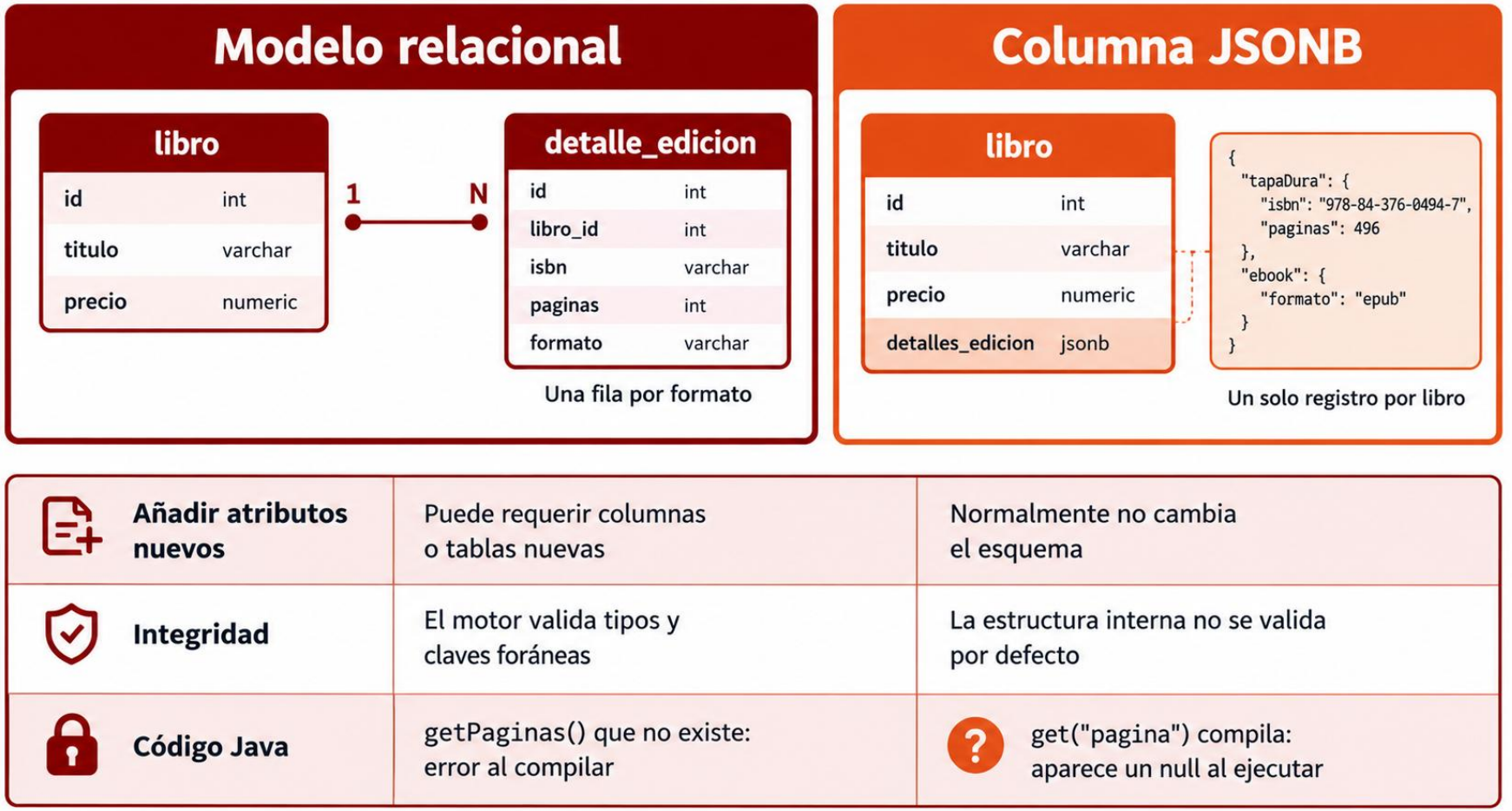
Cómo Hibernate guarda y lee el Map



El tipo Java es un simple Map<String, Object>: no necesitas una clase por cada estructura posible, el mapa acepta cualquier forma de JSON.

Por qué JSONB y no una tabla nueva

Con lo que ya sabes del Tema 1, podrías haber modelado esto como una tabla detalle_edicion con @OneToMany. ¿Por qué elegir JSONB?



Añadir atributos nuevos	Puede requerir columnas o tablas nuevas	Normalmente no cambia el esquema
Integridad	El motor valida tipos y claves foráneas	La estructura interna no se valida por defecto
Código Java	`getPaginas()` que no existe: error al compilar	`get("pagina")` compila: aparece un null al ejecutar

La pérdida de tipado, en la práctica

Tabla relacional

```
detalleEdicion.getPaginas()
// ese método no existe
```

Error de compilación

El código ni llega a ejecutarse: el compilador te avisa antes.

Columna JSONB

```
detallesEdicion.get("pagina")
// era "paginas": un error de una letra
```

Compila sin problema

El fallo aparece al ejecutar: un null inesperado (o un ClassCastException si asumías un tipo).

El compilador ya no te cubre las espaldas: pasa a ser responsabilidad tuya, en tiempo de ejecución.

Índices: sin índice, con GIN y con B-tree



GIN (Generalized Inverted Index) está diseñado para localizar elementos dentro de valores compuestos como JSONB, arrays o texto. El B-tree, en cambio, trabaja sobre valores escalares.

Crear el índice GIN y usarlo

```
CREATE INDEX idx_libro_detalles_edicion
ON libro USING GIN (detalles_edicion);
```

El operador ? permite comprobar si existe una clave en el nivel superior del documento:

```
SELECT * FROM libro
WHERE detalles_edicion ? 'ebook';
```

El índice puede acelerar la consulta, pero PostgreSQL no lo usa siempre: con pocas filas prefiere recorrer la tabla entera. Por eso en la Actividad 2.1 generarás 100.000 filas y compararás el plan de la misma consulta antes y después de crear el índice.

NULL de la columna vs. null dentro del JSON

Tres estados distintos que se confunden con facilidad

Lo que hay guardado	Qué significa	¿Existe la clave ebook?
	El libro no tiene ningún dato de edición, ni siquiera un objeto vacío.	NULL
`{"ebook": null}`	Hay un objeto, la clave existe y su valor es el null de JSON (no el NULL de SQL).	true
`{}`	Hay un objeto, pero no tiene ninguna clave.	false

Ideas clave

1. PostgreSQL es un gestor objeto-relacional: mantiene el modelo de tablas y relaciones, pero incorpora tipos avanzados como JSONB para datos estructurados.

2. JSONB guarda JSON en formato binario indexable: preferible a JSON (texto plano) salvo que necesites preservar el formato exacto.

3. @JdbcTypeCode(SqlTypes.JSON) + @Column(columnDefinition = "jsonb") mapean un Map<String, Object> Java contra una columna jsonb.

4. Objeto simple = valor escalar; objeto estructurado = objeto anidado con posibles colecciones dentro.

Ideas clave (cont.)

1. JSONB gana en flexibilidad de esquema, pero pierde integridad referencial y tipado estricto frente a una tabla relacional: es una decisión de diseño con trade-offs.

2. JSONB no abre ninguna conexión nueva: reutiliza el mismo DataSource del Tema 1.

3. Un índice GIN acelera consultas por el contenido de una columna JSONB; un B-tree no sabe mirar dentro del objeto. Solo gana con volumen y consultas selectivas.

4. NULL de columna, null dentro del JSON y objeto vacío {} son tres estados distintos, con un resultado diferente al comprobar "existe la clave".